

vLLM #51593 Persistent TopK - Negative length causes CTA deadlock

DeepSeek-V4 MTP 在 SM120 下 `persistent_topk` 死锁分析

- <https://github.com/vllm-project/vllm/issues/51593>

辅助 CTA (CTA 1+) 根据正常的 `max_seq_len` 判断没有超长 row, 于是提前退出

主 CTA (CTA 0) 随后读取某个 padding row 的 `length=-1`, 却因 `signed→unsigned` 变成 4294967295, 于是判断是超长 row, 进入需要多个 CTA 协作的 RadixTopK 路径; 主 CTA 到同步点后一直等待那些已经退出的辅助 CTA, 最终形成死锁



Root Cause: 这个问题不是 `shm_broadcast` 或 `multiprocessing queue` 故障, 而是 MTP + FULL CUDA Graph padding 产生一个负的 per-token context length, `persistent_topk` 又把这个 `int32 -1` 当成 `uint32_t` 读取, 变成 `UINT_MAX`, 最终让同一个 CTA group 的 leader 和 peer 对是否需要进入 `multi-CTA radix path` 做出相反判断, 导致 leader 永久卡在 `inter-CTA barrier`。GPU kernel 因此永远不 retire, rank 0 的 output-copy event 一直无法完成, worker response 也就不会 enqueue, 最后 EngineCore 才表现成 `shm_broadcast timeout` 和 `EngineDeadError`

代码块

```
1 vllm serve <path-to>/DeepSeek-V4-Flash \  
2 --tensor-parallel-size 4 \  
3 --attention-backend FLASHINFER_MLA_SPARSE_DSV4 \  
4 --moe-backend flashinfer_cutlass \  
5 --speculative-config '{"method":"mtp","num_speculative_tokens":1}' \  
6 --max-num-seqs 16 --max-num-batched-tokens 8192 \  
7 --max-model-len 65536 --kv-cache-dtype fp8 --block-size 256 \  
8 --gpu-memory-utilization 0.9 --tokenizer-mode deepseek_v4
```

代码块

```
1 CUDA Graph padding slot  
2 -> C4 indexer produces context length -1  
3 -> persistent_topk converts -1 to UINT_MAX  
4 -> CTA1 exits based on the host max_seq_len
```

```
5    -> CTA0 enters the radix path based on UINT_MAX
6    -> CTA0 waits forever for two CTAs at the barrier
7    -> the GPU kernel never retires
8    -> the output-copy CUDA event never completes
9    -> no worker response is enqueued
10   -> EngineCore eventually reports shm_broadcast timeout
```

MTP 是整条故障链的起点。普通 decode 一次 forward 中，每个 request 只处理当前位置的一个 query token，因此 `next_n=1`。这个复现使用 `num_speculative_tokens=1`，所以 target verification 一次会处理两个连续位置，也就是 `next_n=2`：第一个是当前 target position，第二个是 MTP speculative position

- 服务刚开始时可以有很多并发 request，但每个 request 的输出长度不同，有些会先结束，因此 running batch 会随着生成过程逐渐下降。所谓 剩下 3 个真实 request，并不是系统固定创建 3 个 request，而只是某一轮 scheduler 调度时，前面的请求已经陆续结束，此时还有 3 个真实 request 继续生成
- 当剩下 3 个 request 时，MTP 一轮实际上需要处理 $3 \times 2 = 6$ 个真实 query token。但是 FULL CUDA Graph 不能像 eager 模式一样每一步都用任意 shape，它会 replay 已经提前 capture 好的固定 shape。这个场景使用的是 8-token graph，所以 6 个真实 token 必须补到 8 个。额外的两个 token 正好对应一个完整的 MTP request slot，因此 graph metadata 中实际上会出现 4 个 request slot：前三个是真实 request，最后一个只是 padding

代码块

```
1    3 real requests × 2 MTP positions = 6 real query tokens
2                                     ↓
3                FULL CUDA Graph shape = 8
4                                     ↓
5                append 2 padding tokens
6                                     ↓
7                4 request slots
8
9    query_start_loc = [0, 2, 4, 6, 6]
10   seq_lens        = [L0, L1, L2, 0]
11   decode_lens      = [2, 2, 2, 0]
```

- `query_start_loc` 表示每个 request 在 query-token buffer 中的边界，所以前三个 request 分别对应 `[0,2)`、`[2,4)`、`[4,6)`，最后一个 padding request 对应 `[6,6)`，说明它实际上没有任何真实 query token。`seq_lens` 是 request 级别的当前 `sequence length`，padding request 没有历史有效序列，所以它的 `seq_len=0` 是完全正确的；`decode_lens=0` 也同样正确。因此 **CUDA Graph padding 本身没有制造非法数据，真正的 bug 是后面的 MTP metadata preparation 把这个合法的 0 继续展开成负数**
- native MTP path 需要把 request-level `seq_lens` 转成每个 speculative position 对应的 `context length`。对于一个正常长度为 `L` 的 request，一次处理两个连续位置时，第一个位置能够看到 `L-1` 个 KV，第二个位置能够看到 `L` 个 KV，因此代码使用 `seq_lens.unsqueeze(1) - max_decode_len + 1 + offsets` 来生成 `[L-1, L]`。对于正常 request 这个计算没有问题
- padding request 的 `seq_len=0` 仍然被按照固定 `max_decode_len=2` 展开，于是变成 `0 - 2 + 1 + [0,1] = [-1,0]`。也就是说，这里第一次产生真正非法的数据：**context length 语义上表示当前 query 可以访问多少 KV token，它不可能小于 0，padding request 正确结果应该是 `[0,0]`**

代码块

```

1  if use_native and next_n > 1:
2      assert self.decode_seq_lens_buffer.dim() == 1
3      seq_lens_buffer = self.decode_seq_lens_buffer[
4          : num_decodes * max_decode_len
5      ].view(num_decodes, max_decode_len)
6
7      seq_lens_buffer[:] = (
8          seq_lens.unsqueeze(1)
9          - max_decode_len
10         + 1
11         + self.offsets_buffer[:max_decode_len]
12     )
13
14     seq_lens = seq_lens_buffer
15
16     return seq_lens, block_table, decode_lens, num_decodes, requires_padding

```

 普通 decode 的 `next_n=1`，同一个公式退化成 `seq_len - 1 + 1 = seq_len`，所以 padding 的 0 仍然是 0；MTP 的 `next_n=2` 才会让第一个位置相当于 `seq_len-1`，因此当 padding request 的 `seq_len=0` 时得到 `-1`

`variable-length flatten path` 又是另外一种情况：它根据真实 `decode_lens` 去展开 token，padding request 的 `decode_len=0`，因此根本不会生成任何 expanded token，剩余 buffer 还会被清零，所以那条路径不会产生这个 -1

真正有问题的是 native MTP 的固定二维 metadata layout，以及公式本身同样未 clamp 的 uniform speculative path

- DeepSeek-V4 的 `C4 indexer` 还会把这些 `context length` 转换到压缩后的 KV 空间，`compress_ratio=4`。正常长度例如 100 会变成 25，但 Python 的 `//` 是 floor division，所以 `-1 // 4` 仍然是 `-1`，而不是 0。因此这个非法值经过 C4 后没有被消掉，继续向下流入 `top-k`。与此同时，服务配置里的 `max_model_len=65536` 在 C4 indexer 中实际只对应 $65536 / 4 = 16384$ 个 candidate，所以后面 `persistent_topk` 看到的 logits row width 是 16384
- SM120 上 sparse indexer 不会走 `cooperative_topk`，这一架构被 cooperative path 明确排除，最终调用的是 `persistent_topk`。这里真正危险的不是单纯“收到一个 `-1`”，而是 kernel 内部有 **两套决定是否进入 multi-CTA radix 的判断来源**。**`non-leader CTA` 在 kernel 很早的位置只检查 host 已经传进来的 `params.max_seq_len`；当前值是 16384，小于 `RADIX_THRESHOLD=32768`，因此 non-leader CTA 会认为所有 row 都是 short/medium row，没有必要参与 `multi-CTA radix`**

代码块

```
1  FULL CUDA Graph  需要固定 8-token shape
2      ↓
3  3 个真实 MTP requests 只有 6 个 token
4      ↓
5  补出第 4 个 padding request
6      ↓
7  request-level metadata: seq_len = 0  decode_len = 0
8      ↓
9  native MTP 按固定 next_n=2 构造 per-token context lengths
10     ↓
11  0 → [-1,0]
12     ↓
13  DeepSeek-V4 C4 compression  [-1,0] // 4 → [-1,0]
14     ↓
15  非法 -1 进入 persistent_topk
```

- leader CTA 后面处理具体 row 时却不是看这个 host scalar，而是重新从 device memory 读取 `params.lengths[row_idx]`。问题是 `lengths` 的元素本来是 `int32`，但代码直接写成 `const uint32_t seq_len = params.lengths[row_idx]`
- 于是 `padding row` 中的 `-1` 在任何判断之前就变成 `UINT_MAX`，即 4294967295

- leader 接下来比较 `seq_len <= RADIX_THRESHOLD` 时自然得到 false，因此它认为这个 row 是一个超长 row，必须进入 multi-CTA `radix_topk`。这时同一个 CTA group 已经发生逻辑分裂：**peer CTA 根据 `max_seq_len=16384` 提前退出，leader CTA 根据错误的 `row length=UINT_MAX` 进入只有完整 CTA group 才能运行的 radix path**

代码块

```

1  padding request
2      ↓
3  request-level seq_len = 0                ← 合法
4      ↓
5  native MTP per-token expansion
6      ↓
7  [-1, 0]                                  ← 第一次产生非法值
8      ↓
9  C4 compression
10     ↓
11  [-1, 0]
12     ↓
13  persistent_topk
14
15  non-leader CTA                          leader CTA
16      |                                  |
17  params.max_seq_len = 16384              lengths[row] = -1
18      |                                  |
19  16384 <= 32768                          int32 → uint32
20      |                                  |
21  early return                            UINT_MAX
22                                          |
23                                          > 32768
24                                          |
25                                          radix_topk
26                                          |
27                                          waits for peer CTA forever

```



`radix_topk` 进入后使用 `arrival_counter` 做 inter-CTA barrier。当前 `ctas_per_group=2`，所以 barrier 初始阶段需要两个 CTA 都到达，`target` 是 2 但现场 cuda-gdb 抓到的是 `arrival_counter=1`、`target_val=2`：只有 leader CTA 到达过，另一个 CTA 已经执行 early return。因为没有任何 CTA 再能把 counter 从 1 加到 2，所以 leader 永远 spin 在 `wait_ge()`，整个 kernel 永远不会 retire。GPU 因此显示接近 100% utilization，但几乎没有 memory activity；CPU 侧真正接触 GPU 结果的是 async output-copy thread，它一直卡在 `copy_event.synchronize()`，所以 `worker main thread` 看起来只是正常 idle 在 `zmq_poll`

修复首先应该从 producer 做，因为 context length 本身就不应该允许负数。native MTP path 应该在构造 per-token sequence lengths 时直接 clamp 到 0；uniform speculative path 使用同样的算式，也应该同时做 lower-bound clamp。这样 padding request 的 `[-1, 0]` 会直接变成 `[0, 0]`，对所有正常 request 的 `[L-1, L]` 完全没有影响

代码块

```
1  seq_lens_buffer[:] = (  
2      seq_lens.unsqueeze(1)  
3      - max_decode_len  
4      + 1  
5      + self.offsets_buffer[:max_decode_len]  
6  ).clamp_min_(0)
```

第二层应该 harden `persistent_topk`。kernel 不应该假设所有 caller 永远传入合法 length，尤其这个 length 会直接决定内存访问范围和 cooperative topology。不能先把它读进 `uint32_t` 再 clamp，因为 -1 此时已经变成 `UINT_MAX`；应该先保持 signed 类型判断下界，再把合法正数限制到实际 row width

代码块

```
1  const int32_t raw_seq_len = params.lengths[row_idx];  
2  const uint32_t max_valid_len =  
3      min(params.stride, params.max_seq_len);  
4  
5  const uint32_t seq_len =  
6      raw_seq_len <= 0  
7      ? 0u  
8      : min(static_cast<uint32_t>(raw_seq_len),  
9            max_valid_len);
```

- `kernel invariant`：每个 `row` 都满足 $0 \leq \text{seq_len} \leq \min(\text{stride}, \text{max_seq_len})$ 。这样一旦 host 侧确认 `max_seq_len <= RADIX_THRESHOLD`，任何具体 row 也不可能突然大于 `RADIX_THRESHOLD`，从结构上消除 `peer CTA early return`、`leader CTA 进入 radix` 这一整类 deadlock。同时 upper bound 还能防止未来其他 caller 传入超过 logits row width 的错误 length 后产生潜在 OOB read



在 CUDA 里，CTA (Cooperative Thread Array) 基本可以直接理解成一个 CUDA thread block。一个 kernel launch 会启动很多 block，每个 block 内有很多线程；这个 `persistent_topk` 里每个 CTA 有 1024 个线程。普通情况下，一个 CTA 可以独立处理

一个 row，但当 row 很长时，一个 CTA 的计算和 shared memory 不够高效，所以 `persistent_topk` 会把多个 CTA 组成一个 CTA group，共同处理同一个 large row。当前这个现场 `ctas_per_group=2`，意思就是一个 group 里有两个 block：一个是 `cta_in_group=0`，可以叫 leader；另一个是 `cta_in_group=1`，可以叫 peer。两者在 large-row radix 路径中不是各算各的，而是会共同构造 histogram、共享全局状态，并在几个阶段通过 `arrival_counter` 做 barrier 同步。因此只要 leader 进入 multi-CTA radix，peer 就必须也活着并进入同一条路径；否则 barrier 一定等不到

- `persistent_topk` 为省资源做一个优化：如果整个 batch 根本没有 large row，就没必要让 peer CTA 一直存在。因此 kernel 刚开始时会先看一个全局上界 `params.max_seq_len`

代码块

```
1  const uint32_t ctas_per_group = params.ctas_per_group;
2  const uint32_t cta_in_group = blockIdx.x % ctas_per_group;
3
4  if (cta_in_group != 0 &&
5      params.max_seq_len <= RADIX_THRESHOLD) {
6      return;
7  }
```

- `params.max_seq_len` 是 launch 时 host 已经知道的 这个 logits tensor 的最大有效 row width / batch-level upper bound。当前 DeepSeek-V4 C4 indexer 的 logits width 是 16384，而 `RADIX_THRESHOLD=32768`，所以 kernel 得出一个很合理的结论：这个 batch 里理论上不可能有长度超过 32768 的 row。于是 group 里的 peer CTA，也就是 `cta_in_group=1`，直接退出
- leader CTA 留下来，因为 short/medium row 仍然需要它处理
- 每个实际 row 的长度都必须不大于 `params.max_seq_len`。如果全局已经知道最大长度只有 16384，那么后面不应该突然出现一个 row 长度大于 32768。只要这个条件成立，让 peer 提前退出就是完全安全的，因为 leader 后面也绝不可能进入需要 peer 协作的 radix 路径。但是 leader 继续遍历具体 row 时，又会进行第二次判断

代码块

```
1  const uint32_t seq_len = params.lengths[row_idx];
2
3  if (seq_len <= RADIX_THRESHOLD) {
4      // 只有 CTA0 处理 short / medium row
5      ...
6      continue;
7  }
8
```

```
9 // 否则认为是 large row
10 radix_topk(...);-
```

- padding MTP row 的实际 `lengths[row_idx]` 是 `int32` 的 `-1`，但代码却直接用 `uint32_t`

代码块

```
1 const uint32_t seq_len = params.lengths[row_idx];
```

- 因此 `-1` 并没有保持为非法负长度，而是按照二进制补码被解释成 `4294967295`。于是同一个 CTA group 对同一个 batch 得到两个完全矛盾的结论
 - 第一次判断看到 `params.max_seq_len=16384`，所以 peer CTA 认为 没有 large row，可以退出
 - 第二次判断中 leader 看到 padding row 的 `seq_len=4294967295`，所以认为 这是一个超大 row，必须进入 multi-CTA radix
- 进入 `radix_topk` 后，两个 CTA 原本必须在第一个 barrier 汇合。每个 CTA 的 thread 0 会把 `arrival_counter` 加 1，然后等待它达到 `ctas_per_group`

代码块

```
1 if (tx == 0) {
2     red_release(&state->arrival_counter, 1);
3 }
4
5 wait_ge(
6     &state->arrival_counter,
7     (barrier_phase + 1) * ctas_per_group,
8     tx
9 );
```

- 当前 `ctas_per_group=2`，所以正常应该是 leader 加 1、peer 再加 1，counter 到 2 后两边继续执行。但 peer 已经在第一个判断那里 return，因此现在只有 leader 能把 counter 加到 1。cuda-gdb 现场看到的正好就是 `arrival_counter=1`、`target_val=2`



`persistent_topk` 原本允许 peer CTA 根据全局 `max_seq_len` 提前退出，因为它假设具体 row length 永远不会超过这个全局上界；MTP padding 产生的 -1 被错误转成 `UINT_MAX` 后，这个假设被破坏，导致 peer 根据第一个判断退出，而 leader 根据第二个判断进入必须依赖 peer 的 cooperative radix，最终形成不可恢复的 barrier deadlock

在 vLLM 中，`persistent_topk` 主要用于对每个 query 对应的一行候选 logits 进行 **Top-K 选择**，即从大量 KV candidate 中筛选出分数最高的 K 个位置。它并不要求对整行元素进行完整排序，而只需要确定哪些 candidate 属于前 K 名，因此本质上属于 **selection problem**，而不是 full sorting problem。这类操作常见于稀疏注意力、候选路由或大规模 KV 筛选场景中，可以显著减少后续需要参与计算的候选数量

- 对于较长的 logits row，vLLM 采用 **Radix Top-K / Radix Select**。其核心思想是将 FP32 score 转换成保持数值顺序的 32-bit integer key，然后按高位到低位逐步定位第 K 大元素。32-bit key 被划分为 4 个 8-bit 段，每一轮根据当前 8 bit 将候选划分到 256 个 bucket 中，并通过从高 bucket 到低 bucket 的累计计数，判断第 K 大元素落在哪个 bucket。确定该 bucket 后，只保留具有相同高位前缀的候选，在下一轮继续检查更低的 8 bit。经过 4 轮后，就能够得到第 K 大元素对应的完整 32-bit key，即 Top-K 的边界值 `pivot`

代码块

```
1  struct PersistentTopKParams {
2      const float* input;          // [num_rows, stride]
3      int32_t* output;             // [num_rows, top_k]
4      const int32_t* lengths;      // [num_rows]
5
6      RadixRowState* row_states;
7
8      uint32_t num_rows;
9      uint32_t stride;
10     uint32_t top_k;
11
12     uint32_t chunk_size;
13     uint32_t ctas_per_group;
14     uint32_t max_seq_len;
15 };
16
17 struct RadixRowState {
18     uint32_t histogram[3][256];
19     uint32_t remaining_k;
20     uint32_t prefix;
21     int arrival_counter;
22     int output_counter;
23 };
```

因此，这个算法的关键并不是生成一个完全有序的 Top-K 序列，而是高效地确定 Top-K 分界点。所有分数严格大于 `pivot` 的元素必然属于 Top-K，而与 `pivot` 相等的元素只需要选取足够数量以补满 K 个结果。相比对长度为 N 的 logits 完整排序，Radix Select 只需要进行少量固定轮次的线性扫描和 histogram 统计，更适合 GPU 上处理数万甚至更长的候选序列

- 在 vLLM 的实现中，超长 row 还会被划分给多个 CTA 并行处理。每个 CTA 负责一段 logits，独立统计局部 256-bin histogram，再将结果合并成整行的全局 histogram；随后所有 CTA 基于相同的统计结果共同确定下一轮的 radix prefix。因而，这套实现可以概括为：****将长 logits row 分片并行扫描，通过 4 轮 8-bit histogram 逐步确定第 K 大元素的数值边界，再根据该边界输出最终 Top-K candidate indices。****它的主要价值在于避免全排序，并充分利用 GPU 的并行 histogram 和多 CTA 协同能力来降低大规模 Top-K 选择的成本

代码块

```

1 CTA shared memory
2
3 ┌───────────────────────────────────────────┐
4 │ local_histogram[256]                    │ 当前 CTA 自己 chunk 的 histogram
5 └───────────────────────────────────────────┘
6 │ suffix_sum[256]                        │ 合并后的 histogram 做 suffix scan
7 └───────────────────────────────────────────┘
8 │ shared_scalars[5]                      │
9 │   [0] prefix                          │
10 │   [1] remaining_k                     │
11 │   [2] selected_bucket                 │
12 │   [3] next_remaining_k               │
13 │   [...]                             │
14 └───────────────────────────────────────────┘
15 │ shared_ordered[chunk_size]            │ 本 CTA 负责的 FP32 ordered keys
16 └───────────────────────────────────────────┘

```